Conan: A Hallucination-Resistant RAG Pipeline for Egyptian Criminal Law in Arabic

Conan: A Hallucination-Resistant Retrieval-Augmented Generation Pipeline for Egyptian Criminal Law in Arabic

Authors: Mirna Nageh, Wageh Mostafa, Seif Sherif, Mariam Mohamed, Mayar Mohammed

---

Abstract

Retrieval-Augmented Generation (RAG) has become the de-facto architecture for grounding large language models (LLMs) on domain-specific knowledge, yet its application to Arabic legal information retrieval continues to suffer from two well-documented failure modes: hallucinated statutory citations and the LLM's tendency to confabulate when retrieval surfaces topically-related but legally-inadequate context. In this work we present **Conan**, a production-grade Arabic legal assistant for the Egyptian Criminal Code (قانون العقوبات) and the Code of Criminal Procedure (قانون الإجراءات الجنائية). Starting from a strong hybrid-retrieval baseline (FAISS dense search + BM25 sparse search fused via Reciprocal Rank Fusion, with cross-encoder reranking), we introduce a multi-layered grounding pipeline that drives the rate of hallucinated article-number citations from a baseline of 28.6 % down to 0.0 % on a 28-question Arabic legal benchmark. Our contributions span six interlocking mechanisms: (i) an input-gating filter that rejects fragmentary user inputs before retrieval, (ii) a strict citation-grounding prompt regime separating substantive from procedural law, (iii) a corrective LLM retry with an explicit block-list of forbidden article numbers, (iv) an **article-lookup rescue** that indexes 1,494 distinct Egyptian-law articles from chunk metadata and uses them as a second-chance grounding source, (v) deterministic answer post-

processing that rewrites awkward template-leakage phrases and strips persona-breaking openers, and (vi) **iterative retrieval** that auto-expands the k-window when validation fails — without requiring an additional LLM call. We further introduce a document-upload subsystem supporting three retention semantics — per-question attachment, session-attached document, and permanent corpus ingestion — covering `.txt`, `.pdf`, and `.docx` formats. The complete system, evaluated on a 28-question benchmark covering substantive and procedural Egyptian criminal law, achieves a **100 % evidence-validation pass rate with zero hallucinations**, an 8-fold improvement over the baseline. We release the system, the evaluation harness, and a five-version CSV trail documenting the incremental improvements.

Keywords: Arabic Natural Language Processing; Retrieval-Augmented Generation; Legal Information Retrieval; Hallucination Mitigation; Egyptian Criminal Law; Article-Lookup Rescue; Iterative Retrieval

---

1. Introduction

The Arabic language is the official tongue of more than twenty sovereign states and the first language of an estimated 422 million speakers, yet it remains comparatively underserved by modern natural language processing (NLP) systems [1]. This under-representation is especially acute in *Arabic legal* NLP, where the combination of Modern Standard Arabic (MSA), domain-specific terminology, dialectal variation, and the morphological richness of the language compound the standard challenges of grounded language modelling.

Retrieval-Augmented Generation (RAG) has emerged as the dominant architecture for closing the gap between a frozen LLM's parametric memory and the dynamic, citation-heavy demands of legal practice. By retrieving relevant passages from a curated corpus and conditioning the generator on those passages, RAG systems can in principle produce answers that are both fluent and traceable [2,6]. However, recent surveys of Arabic legal RAG implementations — including the Moroccan family-code study of Hrimech et al. [2] and the morphologically-aware retrieval study of Aboasal et al. [5] — identify a recurring failure pattern: the LLM remains liable to **hallucinate statutory citations**, even when grounded retrieval is available, particularly when the user's question maps to articles that were not surfaced in the retrieval top-k.

This paper presents **Conan** ("كونان"), an Arabic legal-AI assistant for Egyptian Criminal Law. Conan is built on a hybrid-retrieval foundation (FAISS dense embeddings + BM25 sparse search, fused via Reciprocal Rank Fusion, with cross-encoder reranking) and extends that baseline with a *grounding-defence pipeline* designed specifically to suppress the citation-hallucination failure mode documented in the Arabic-legal-RAG literature. We argue that no single technique is sufficient — instead, a **layered** approach combining input filtering, prompt engineering, post-hoc validation, article-level metadata lookup, and answer rewriting is required.

1.1 Contributions

The contributions of this work are as follows:

1. **A multi-stage grounding pipeline** for Arabic legal RAG that reduces the rate of hallucinated article-number citations from **28.6 % to 0.0 %** on a 28-question Egyptian criminal-law benchmark, an absolute reduction of 28.6 percentage points across six measurement points (v3 → v8).
2. **An input-gating mechanism** (Section 4.1) that detects and rejects fragmentary, markdown-formatted, or otherwise incomplete user inputs before they incur an LLM call — a class of input we observed accounted for approximately one in five fabricated-citation cases in baseline measurements.
3. **An article-lookup rescue subsystem** (Section 4.4) that indexes 1,494 distinct Egyptian-law article numbers from chunk metadata at startup and uses them as a deterministic second-chance grounding source when post-generation evidence validation flags a missing citation. The mechanism converts validated hallucinations into grounded answers *without* re-running the costly retrieval stage.
4. **Iterative retrieval with adaptive k-expansion** (Section 4.6) that automatically widens the retrieval window from k=7 to k=14, then k=21, when evidence validation fails — re-using cached chunks where possible and avoiding additional LLM calls. This mechanism captures legally-relevant articles that the initial reranker's top-k missed.
5. **A document-upload pipeline** (Section 5) supporting three distinct retention semantics — per-question, session-attached, and permanent-corpus — across the three most common Arabic document formats (`.txt`, `.pdf`, `.docx`) and pasted-text strings. The pipeline shares a single parser and Arabic-cleaning module to guarantee consistent behaviour across modes.

6. **A five-version evaluation trail** (Section 6) on a 28-question Arabic legal benchmark, with the per-version CSV outputs released alongside the system, allowing reproducible auditing of every architectural change.
7. **A procedural-defence reasoning checklist** (Section 4.8) that encodes expert-lawyer case-analysis heuristics — warrant-timeline nullity, identifier mismatch, chain-of-custody, and intent-from-profession — into the case-analysis prompts, paired with a *grounding-safe* few-shot exemplar that raises reasoning coverage without reintroducing hallucinated citations.

The remainder of the paper is organised as follows. Section 2 surveys related work on Arabic legal RAG, multilingual retrieval, and hallucination mitigation. Section 3 describes the base retrieval architecture. Section 4 presents the six grounding-defence mechanisms in detail. Section 5 describes the document-upload subsystem. Section 6 reports the evaluation methodology and results. Section 7 discusses limitations and Section 8 concludes.

---

2. Related Work

2.1 Arabic RAG: foundations and challenges

El-Beltagy and Abdallah [1] present a foundational case study of Arabic RAG, evaluating multiple semantic-embedding models and LLMs in the retrieval and generation stages respectively, and explicitly investigating the impact of dialectal variation between document language and query language. Their work establishes the baseline architectural template that the present paper inherits — a semantic retriever feeding a generator LLM — while also flagging the central challenge of selecting an embedding model that captures the semantic nuances of Arabic without over-relying on the English-dominated pre-training data of most multilingual embedding models.

Alghamdi et al. [4] focus specifically on the retriever component, evaluating multiple Arabic information-retrieval techniques in the context of question answering. They demonstrate that retrieval quality is the dominant factor in downstream answer correctness — a finding our own iterative-retrieval mechanism (Section 4.6) extends by treating retrieval k as an adaptively-tunable parameter rather than a fixed hyperparameter.

2.2 Arabic legal RAG

Hrimech et al. [2] present the most direct prior work to ours: a RAG system for the *Moroccan* family code, built on the BGE-m3 multilingual embedding model with a custom dataset of 2,500 Arabic question–answer pairs. Their evaluation, using Mean Reciprocal Rank, Recall@k, F1, and a panel of semantic-fidelity metrics, finds that BGE-m3-driven RAG substantially outperforms standalone LLMs on legal-Q&A — but they explicitly call out the challenges of *legal-terminology adherence*, *content-validity of reproduced clauses*, and the *scarcity of annotated Arabic legal corpora*. The grounding-defence mechanisms introduced in the present work (Sections 4.1–4.5) target these specific concerns in the context of Egyptian criminal law.

Aboasal et al. [5] approach the same domain from a *morphological-and-semantic* angle, introducing a synthetic benchmark of 500 legal articles and 1,000 Q&A pairs and evaluating the impact of Farasa-based morphological segmentation on BM25, Ada v3, BGE-M3, GTE, and Mistral-embed retrievers. Their hybrid Farasa-BM25 + Ada v3 configuration reaches a Mean Average Precision of 0.8304 and nDCG@10 of 0.8626 — figures that informed our choice to keep BM25 as a complementary signal alongside dense FAISS retrieval (Section 3) rather than abandoning it in favour of pure dense retrieval.

2.3 Benchmarking Arabic legal reasoning

Abu Shairah et al. [3] introduce **ALARB**, an Arabic Legal Argument Reasoning Benchmark comprising over 13,000 commercial court cases from Saudi Arabia, with each case annotated with the facts, the court's step-by-step reasoning, the verdict, and the cited regulatory clauses. ALARB sets the methodological precedent for our own evaluation harness: a benchmark whose primary axis is not the linguistic *fluency* of the generated text but its *legal-grounding fidelity* — specifically, whether cited statutory articles can be traced to retrieved or otherwise-available source text. While our 28-question benchmark is far smaller in scale, it inherits the philosophy that *what an LLM cites matters more than how it phrases the citation*.

2.4 RAG fundamentals and tooling

A pragmatic introduction to RAG architecture is provided by the Mini-RAG documentation [6], which decomposes the pipeline into three steps — *retrieval* (semantic search over a knowledge base), *augmentation* (passage injection into the prompt), and *generation* (LLM completion conditioned on the augmented prompt) — and walks through chunking, indexing, and query workflows. While not a peer-reviewed publication, the document is representative of the implementation-

oriented tutorial literature that informed many practical design decisions in our system (chunking strategy, vector-database choice, prompt scaffolding).

2.5 Positioning of the present work

Relative to [1] and [2], our work shifts the locus of grounding from the retrieval stage alone to a *post-generation defence pipeline*: even with state-of-the-art retrieval, an LLM may emit a memory-based citation that retrieval did not provide, and our evaluation shows that detecting and rescuing this case is at least as important as improving the retriever. Relative to [4] and [5], we share the focus on hybrid retrieval but extend the operational regime to include *iterative k-expansion* and *article-number-keyed rescue*. Relative to [3], we adopt the grounding-fidelity evaluation philosophy but apply it to an Egyptian-criminal-law corpus rather than a Saudi-commercial-law one.

---

3. System Architecture

Conan is implemented as a FastAPI service backed by a retrieval index of 47,028 chunks built from the Egyptian Penal Code, the Code of Criminal Procedure, a curated subset of cassation rulings, the Cassation Encyclopedia, and supplementary criminal-law references. The base architecture follows the pattern established by [1], [2], and [4]:

Retrieval. A user query is encoded with a multilingual sentence-transformer (paraphrase-multilingual-MiniLM-L12-v2 in our deployment) and used to query a FAISS index of pre-computed chunk embeddings. In parallel, the same query is BM25-tokenised (with Arabic-Indic digit normalisation and alef-form unification) and scored against the corpus via a `rank_bm25` BM25Okapi index. The top-30 results from each branch are fused using **Reciprocal Rank Fusion** with the canonical RRF constant `k = 60`. The fused top-30 candidates are passed to a **cross-encoder reranker** (BGE-Reranker-v2-M3 with sigmoid-bounded outputs), which produces the final top-k set (default `k = 7`) used as context.

Context expansion. Each surviving chunk is grown by one *same-source* neighbour on each side, providing roughly 2× the textual context per chunk while preventing cross-document contamination.

Generation. The final context, expert-rule entries (matched by keyword from a curated `expert_rules.json`), and the user's question are formatted into a prompt template enforcing strict textual adherence ("Use the provided texts ONLY") and

dispatched to the LLM provider chain. The deployment uses Groq's `llama-3.3-70b-versatile` as the primary, with Google Gemini and OpenRouter Qwen as fallback tiers. Multi-key rotation handles per-minute rate limits.

Confidence scoring. A weighted heuristic over four signals — mean rerank score, source count (capped at 5), article-validation pass/fail (Section 4.3), and topic-match (encyclopedia folder taxonomy matching) — produces a confidence value in [0, 1] alongside a structured breakdown. Answers below a configurable threshold (default 0.5) carry a low-confidence warning.

Session management. Multi-turn conversations are managed via a thread-safe `SessionManager` with sliding-window compaction, atomic JSON-file persistence, and a TTL-based pruner. Each session can additionally carry a list of *attached documents* (Section 5).

---

4. Grounding-Defence Pipeline

The core contribution of this paper is the layered defence pipeline that sits on top of the architecture in Section 3. Each layer targets a specific failure mode observed in baseline measurements (Section 6.1).

4.1 Input gating

Failure mode. In baseline evaluation, fragmentary user inputs (markdown headings such as `## المستوى الأول`, single-word bullets such as `* الجناية`, or numbered list-items ending in a colon such as `1. ما المقصود بمبدأ:`) frequently triggered the LLM to *fabricate* a plausible-sounding question and answer it. These cases accounted for 5 out of 28 (17.9 %) hallucination instances in baseline measurements.

Mechanism. A pure function `is_meaningful_query` evaluates the user's input against a sequence of regex-based predicates: minimum Arabic-character count, leading markdown-header detection, leading list-bullet detection, leading numbered-fragment detection, and a trailing-colon check that rejects sentences that are clearly anticipating a continuation (e.g., `ما المقصود بمبدأ:`). Inputs that fail any predicate are short-circuited with a 9-millisecond response containing a polite Arabic prompt to reformulate the question (`يبدو أن السؤال غير مكتمل أو غير واضح. يرجى`

صياغة سؤال قانوني كامل …). The input gate is **bypassed** when an attached document is present (Section 5), since "لخّص" + a file is a meaningful request even though "لخّص" alone is not.

Result. In the v8 evaluation, the input gate caught and rejected 8 of 28 inputs (28.6 %) without incurring an LLM call. The latency reduction on these inputs is roughly 3,000× compared to the full pipeline.

4.2 Citation-grounding prompts and law-naming disambiguation

Failure mode. Two distinct prompt-level failure modes were observed: (a) the LLM cites article numbers from training memory that *appear* plausible but are not in the retrieved context; (b) the LLM confuses substantive (Penal Code) with procedural (Code of Criminal Procedure) law, citing the wrong code for a given query. For example, baseline answers to questions about pre-trial detention (التوقيف الاحتياطي) frequently cited Article 300 *of the Penal Code* — when Article 300 of the Penal Code does not address that topic; the relevant article is in the *Code of Criminal Procedure*.

Mechanism. Both `qa_restrictive` and `chat` prompts are extended with explicit citation-grounding rules:

- *Citation grounding (absolute):* "Cite an article number ONLY if those exact digits appear in the Context below."
- *Law-naming disambiguation:* procedural matters (التوقيف الاحتياطي، التحقيق، التفتيش، الطعن، النقض) must be cited from قانون الإجراءات الجنائية; substantive matters (تعريف الجرائم، أركانها، عقوباتها) from قانون العقوبات.
- *No casual openings:* explicit prohibition on تمام, بالتأكيد, حسناً, and similar persona-breaking conversational fillers.

The mechanism follows the spirit of the citation-validity recommendations of Hrimech et al. [2], who emphasise the importance of "content validity of legal clauses reproduced from retrieval systems."

4.3 Evidence validation

Mechanism. Following the legal-grounding-fidelity philosophy of [3], every LLM output is post-processed by an evidence validator. A regex-based extractor

(`extract_article_references`) — operating on Arabic-Indic-digit-normalised text — identifies every article number cited in the answer. The set of cited articles is compared against the set of article numbers extracted from the *retrieved context* (concatenated). Articles cited but not present in the context are flagged as **missing**, and the answer is marked as failing validation. The same extractor is used on both sides (answer and context) to avoid spurious mismatches caused by plural-form variations (المواد 211، 212، 213 vs المادة 211).

4.4 Corrective retry with forbidden-article block-list

Mechanism. When evidence validation flags missing articles, the system re-prompts the LLM with a *correction* prompt (`qa_retry_ungrounded`) that includes the original draft answer and an *explicit list* of article numbers the LLM must NOT cite (because they are known not to be in the context). The retry is run at most `RETRY_MAX_ATTEMPTS` times (default 1 — a careful tuning informed by the Groq free-tier 6,000 tokens-per-minute budget, beyond which a second retry cascades into 429 rate-limit errors). The retry's output is accepted only if it strictly improves the missing-article count.

4.5 Article-Lookup Rescue

Failure mode. Even after retry, a class of hallucinations persisted: the LLM would cite an article that *did* exist in the dataset, just not in the top-k chunks for that specific query. Inspection of the chunk metadata revealed that 14,882 of 47,028 chunks (31.6 %) carried `referenced_articles` metadata, covering 1,494 distinct article numbers — including, for instance, 34 chunks referencing Article 87 and 14 chunks referencing Article 300, both articles the baseline LLM cited "from memory."

Mechanism. At startup, `ArticleLookupService.load()` performs a single pass over `chunks.pkl` and builds an index `{article_number → [chunk_index, …]}`. Chunks are ranked within each article's list by a three-tier quality score:

1. **Document-type tier**: `penal_code` and `criminal_procedure` (the actual law text) > `cassation_ruling`, `criminal_law_reference`, `legal_rules_collection` > `cassation_encyclopedia`, `legal_reference`. This prevents, for example, Rome-Statute (ICC) passages from the Encyclopedia of the Counsellor from being preferred over chunks from the Egyptian code.

2. **Article focus**: chunks referencing *fewer distinct articles* are preferred, on the heuristic that they are more focused on the queried article specifically rather than mentioning it as a passing cross-reference.
3. **Text length**: longer chunks are preferred as a final tiebreaker, on the assumption that they are more likely to contain a complete article definition.

When post-retry evidence validation still flags missing articles, the rescue path is triggered: the top-2 chunks for each missing article are retrieved, concatenated into a `rescue_context` block, and a *third* LLM call is made with the `qa_rescue_with_lookup` prompt. This prompt explicitly forbids the LLM from inventing *neighbouring* articles (e.g., if the references contain Article 87, the LLM must not write "88, 89, 90, 91, 92" as a sequential list unless each of those numbers also appears in the provided text) and explicitly forbids citing a passage that is about a *different law or topic* than the user's question.

The rescued answer is validated against the *combined* context (original retrieval + rescue chunks). It is accepted only if grounding improves; otherwise the original answer is preserved with a hallucination warning.

Result. Three of the four remaining hallucinations after the v4 retry pipeline (specifically Q18 citing Articles 30/31, the cases corresponding to Articles 122/123 of the Penal Code, and the case for Article 134) were converted to passing answers by the article-lookup rescue. Q18 went from a fail (in v3 through v5) to a pass at 48 % confidence in v6 with the explicit warning تم استرجاع مواد إضافية من قاعدة البيانات للتحقق من الاستشهادات (مواد: 30, 31).

4.6 Iterative Retrieval (Adaptive k-Expansion)

Mechanism. Following the spirit of [4]'s recommendation that retrieval quality is the dominant determinant of downstream correctness, we treat the retrieval window-size k as an *adaptive* parameter rather than a fixed hyperparameter. After the initial generation and evidence validation, if validation flagged missing articles AND `USE_ITERATIVE_RETRIEVAL` is enabled (default), the system re-runs *only* the retrieval stage — not the LLM — at successively larger k values from the `ITERATIVE_K_SEQUENCE` (default `[7, 14, 21]`). Any new chunks not already in the context are appended, and the validator is re-run on the combined context. If any of these widened windows surfaces chunks containing the missing articles, validation passes *for free* (no LLM call). Only if this cheap expansion still fails does the corrective retry (Section 4.4) and rescue (Section 4.5) take over, but now with a wider context already in hand.

This mechanism has favourable cost characteristics: easy queries (those that pass at the initial k) pay nothing extra; hard queries pay 2–3 cheap retrieval cycles (~1 s each on CPU) before incurring an LLM call. A user-facing Arabic warning (تم توسيع نطاق البحث تلقائياً إلى k مرجعاً) is surfaced whenever the expansion fires.

4.7 Answer Post-Processing

Mechanism. A deterministic, regex-driven post-processing layer (`postprocess_answer`) executes after the final answer (post-retry, post-rescue) is selected and before it is returned to the user. Two transformations are applied:

1. **Casual-opener stripping.** A curated list of conversational fillers (حسناً, بالتأكيد, تمام, طبعاً, سوف أجيب, لقد قرأت السؤال, بالنسبة لسؤالك, ...) is stripped from the answer's leading position if present. The list also includes longer multi-word openers (سوف أجاوب على أسئلتك بتفصيل) ordered longest-first to prevent stranded fragments.
2. **Refusal-phrase normalisation.** When the LLM splices the template-refusal phrase المادة المطلوبة غير متوفرة في السياق المقدم mid-clause (a frequent observation in pre-v6 outputs, e.g., وفقاً للمادة المطلوبة غير متوفرة في السياق المقدم، يبدو أن السؤال غير مكتمل), the regex `_EMBEDDED_REFUSAL_RE` matches the awkward construction — accounting for Arabic prefix-contraction rules (ل + ال → لل) — and rewrites it as a standalone sentence (النصوص المقدمة لا تتضمن المادة المطلوبة.).

A separate classifier `looks_like_refusal` detects whether the post-processed answer is dominated by refusal markers; if so, a `is_refusal` warning is surfaced. Confidence is not capped automatically (an earlier design choice we reverted in v5 after observing it dragged down the mean on borderline partial-but-valid answers).

4.8 Procedural-Defence Reasoning Checklist (Case Analysis)

Failure mode. The grounding mechanisms in Sections 4.1–4.7 ensure that what the system *cites* is correct; a complementary failure mode concerns what it *omits*.

The case-analysis endpoints (`/weakness`, `/defense`, `/forensic`) consume a full criminal case file rather than a single question, and a domain-expert review of their output scored the legal reasoning at roughly 70 %: the analyses were well-grounded but missed several high-value procedural-nullity and criminal-intent arguments that a practising Egyptian criminal-defence lawyer applies routinely — and in one case *over-claimed* a defect that did not exist.

Mechanism. We encode the expert feedback as a **procedural-defence checklist** of five reasoning procedures, appended to the system prompts of the three case-analysis endpoints and reinforced in their user templates:

- **A. Timeline vs. prosecution warrant** — compare the exact arrest/search time against the time the prosecution warrant (إذن النيابة) issued; a seizure preceding the warrant voids the arrest and everything built on it (ما بُني على باطل فهو باطل). The rule explicitly instructs the model to read 12:00 الساعة صباحاً as the *start* of the day, correcting a clock-reading error observed in baseline output.
- **B. Identifier mismatch** — compare every identifier in the warrant (vehicle plate numbers, names, IDs) against the seizure record; any discrepancy places the seized item outside the warrant and may evidence تلفيق.
- **C. Territorial jurisdiction** — apply correctly and do *not* over-claim spatial excess for a location inside the precinct. This point is a precision correction (a false-positive suppressor), not an additional-recall rule.
- **D. Chain of custody** — challenge the integrity of the sealed exhibits (التحريز) when the officer who sealed them differs from the one who wrote the seizure record.
- **E. Intent from profession** — weigh the defendant's occupation against the nature of any seized cash to contest trafficking intent (انتفاء قصد الاتجار).

The checklist is paired with a single fully-worked, *fictional* few-shot exemplar demonstrating all five points end-to-end. Crucially, the exemplar is **grounding-safe by construction**: it names defence doctrines (بطلان القبض والتفتيش, انتفاء قصد الاتجار) but contains *no* المادة `N` article number, so it cannot teach the model to emit an ungrounded citation that the evidence validator (Section 4.3) would flag — raising reasoning coverage while preserving the 0 % hallucinated-citation

property. Each checklist item is explicitly conditioned on the facts supporting it ("raise a point only when the facts genuinely support it — never invent one"), so the additions trade no precision for their gain in recall.

A companion **charge-fidelity** rule extends the same precision principle to the offences themselves: the case-analysis endpoints are constrained to enumerate only offences actually charged or described in the case file, and are explicitly forbidden from inventing an uncharged offence (e.g., adding قيادة بدون رخصة when the file never mentions a licence). This closes a fabrication mode observed during live validation, where the generator appended a plausible-but-uncharged offence to an otherwise grounded analysis.

---

5. Document Upload Pipeline

The system supports three distinct retention semantics for user-supplied documents, addressing different operational use cases observed during user testing.

5.1 Per-Question Attachment (`POST /api/v1/qa/upload`)

A `multipart/form-data` endpoint that accepts a `question` field, an optional `file` field (`.txt` / `.pdf` / `.docx`), and an optional `text` field (raw string). The attached content is parsed (Section 5.4), wrapped in `[المستند المرفق]` delimiters, and *prepended* to the retrieved legal context for **this request only**. The attachment is not persisted and does not enter the permanent index. Article numbers cited in the answer can come from *either* the attachment or the retrieved corpus — the evidence validator (Section 4.3) operates on the union. The input gate (Section 4.1) is skipped when an attachment is present.

5.2 Session-Attached Document (`POST /api/v1/chat/attach`)

A multipart endpoint that *binds* a document to a session, persisting it alongside the conversation history. Each subsequent chat turn prepends *every* attachment to the LLM context (via `Session.format_attachments()`, which applies a per-document length cap to bound the context budget). Attachments survive server restarts via the existing JSON session-persistence machinery. Companion endpoints `GET /chat/{sid}/attachments`, `DELETE /chat/{sid}/attachments/{doc_id}`, and `DELETE /chat/{sid}/attachments` provide the full lifecycle.

5.3 Permanent Corpus Ingestion (`POST /api/v1/ingest`)

The pre-existing ingest endpoint, surfaced in the Streamlit UI through this work. Uploaded documents are chunked using the doc-type-aware chunking configuration (`CHUNKING_CONFIGS`), embedded via the same embedding model used at startup, and merged into the FAISS and BM25 indices on disk. The retrieval service is then hot-reloaded so subsequent queries see the new data without a server restart.

5.4 Shared Parsing Layer (`POST /api/v1/parse`, `services/upload_helper.py`)

To avoid behavioural divergence across the three modes, all parsing flows through a single helper module. The parser dispatches by file extension:

- `.txt`: read with a fallback sequence of Arabic encodings (UTF-8 → CP1256 → ISO-8859-6);
- `.pdf`: extract via `pdftotext -layout -enc UTF-8` (poppler-utils) when available, falling back to PyPDF2;
- `.docx`: extract via `docx2txt`;
- `.doc` (legacy binary): explicitly rejected with a helpful Arabic error directing the user to save as `.docx` or `.pdf`.

All extracted text is Arabic-cleaned (diacritic removal, Arabic-Indic digit normalisation, alef-form unification, page-number stripping). A 100,000-character cap bounds memory; a 30-character minimum filters empty or OCR-failed uploads. A `/parse` endpoint exposes this layer directly, allowing UI clients to preview and edit the parsed text before submitting it for analysis.

---

6. Evaluation

6.1 Benchmark

We constructed a 28-question Arabic legal benchmark spanning three difficulty tiers (basics, application, advanced) and covering the substantive (Penal Code) and procedural (Code of Criminal Procedure) law domains. The benchmark deliberately includes:

- **5 well-formed substantive questions** (e.g., penalty for armed robbery, conditions of legitimate self-defence, elements of premeditated murder);

- **1 procedural question** (شروط التوقيف الاحتياطي) that historically triggered wrong-law confusion in baseline systems;
- **5 input-gate test cases**: markdown headings (## المستوى الأول), bullet fragments (* الجناية, * الجنحة, * المخالفة), and colon-terminated incomplete sentences (1. ما المقصود بمبدأ:);
- **17 application and reasoning questions** drawn from undergraduate criminal-law curricula, ranging from simple case analyses to comparative questions (ما الفرق بين القتل العمد والقتل الخطأ؟).

Each question is evaluated against two metrics: (i) **evidence-validation status** — whether every article number cited in the answer can be traced to the retrieved (or rescued) context, and (ii) **confidence score** as computed by Section 3's heuristic. The primary axis of evaluation is hallucination rate: the percentage of questions for which evidence validation fails.

6.2 Evaluation Protocol

A CLI harness (`run_eval.py`) posts each of the 28 questions to `POST /api/v1/qa` with `k = 7` and writes the per-question result (question, first 150 characters of answer, confidence %, validation status, warning count, source count, API/total/retrieval latency) to a CSV. A 4-second pause separates consecutive requests to keep the Groq free-tier 6,000-tokens-per-minute budget refilled. A second harness (`test_qa.py`) provides a human-readable per-question report with pass/fail/refusal classification and an exit code suitable for CI integration.

6.3 Results

Table 1 summarises the headline metrics across five system versions corresponding to the cumulative introduction of each grounding-defence layer.

Table 1: Evaluation across five system versions (28-question benchmark)

Metric	v3 baseline	v4	v6	v7	v8
Passed evidence validation	20/28 (71.4 %)	26/28 (92.9 %)	27/28 (96.4 %)	27/28 (96.4 %)	**28/28 (100.0 %)**

Metric	v3 baseline	v4	v6	v7	v8
Hallucinated citations	8/28 (28.6 %)	2/28 (7.1 %)	1/28 (3.6 %)	1/28 (3.6 %)	**0/28 (0.0 %)**
Average confidence	41.2 %	38.0 %	39.8 %	40.0 %	41.9 %
Input-gated (no LLM call)	0	8	8	8	8

The progression of hallucination rate across versions corresponds to the cumulative introduction of each layer:

- **v3 → v4**: input gate + corrective retry + tightened prompts → **−21.5 percentage points** (8 → 2 hallucinations).
- **v4 → v6**: article-lookup rescue → **−3.5 percentage points** (2 → 1).
- **v6 → v7**: rescue prompt tightening + quality-tiered chunk ranking → **0 net change** in pass rate, but the *specific* failing question shifted (LLM non-determinism).
- **v7 → v8**: LLM upgrade from `llama-3.1-8b-instant` to `llama-3.3-70b-versatile` → **−3.6 percentage points** (1 → 0).

We note that the v5 measurement (multi-query synonym expansion) is omitted from Table 1 because that mechanism was found to *regress* performance on the current index: synonym expansion diluted retrieval quality by pulling in less-relevant chunks. The multi-query implementation is retained in the codebase but disabled by default (`USE_MULTI_QUERY=False`) — a useful negative result we report in detail in Section 7.

6.4 Qualitative Observations

Two questions are particularly informative:

- **Q3 — التوقيف الاحتياطي.** In v3, this question triggered the wrong-law failure mode: the LLM cited Article 300 of the Penal Code. After the prompt-level disambiguation in v4, the LLM correctly identifies the question as procedural and produces a refusal-style answer when retrieval does not surface the relevant Code of Criminal Procedure chunks. Validation passes (no

fabricated citation), at 59 % confidence with a refusal-detection warning surfaced.

- **Q18 — هل تعتبر الجريمة تامة أم شروع؟** This question proved the most persistent failure: it failed in v3 (at 53 % confidence), regressed under v4's pure-prompt approach (citing Articles 30 and 31), and recovered to passing only with the article-lookup rescue in v6 (تم استرجاع مواد إضافية من قاعدة البيانات للتحقق من الاستشهادات (مواد: 30, 31), 48 % confidence). The v7 stricter rescue prompt initially introduced a regression on this question (the LLM hallucinated a fresh Article 32 instead) which the v8 LLM upgrade resolved.

These observations underline a key point: **no single mechanism is sufficient**. The v3-baseline failures distribute across distinct root causes (input fragments, prompt ambiguity, retrieval coverage gaps, LLM memory citations), and each requires its own layer of defence. The cumulative pipeline reduces every category to zero in v8, but each layer contributes meaningfully and removing any one would degrade performance.

---

7. Limitations and Future Work

LLM cost on free tiers. The full pipeline can incur up to 3 LLM calls per hard question (initial generation, corrective retry, article-lookup rescue). Combined with the Groq free-tier 6,000-tokens-per-minute budget, this places a practical ceiling on evaluation throughput. We mitigate via inter-request delays in the evaluation harness and a hard cap of `RETRY_MAX_ATTEMPTS = 1` on the corrective retry. Paid-tier providers (OpenRouter GPT-4o / Anthropic Claude) would lift this constraint at a marginal cost increase.

Multi-query expansion regression. The deliberate negative result reported in v5 — that heuristic synonym expansion *reduces* pass rate on the current index — is consistent with the [1] observation that embedding-model semantics for Arabic do not translate cleanly across closely-related but lexically-distinct legal terms (e.g., التوقيف الاحتياطي vs الحبس الاحتياطي). A semantically-aware query expansion using the LLM itself, rather than a static synonym table, may recover this lost potential — a direction for future work.

Article-lookup precision. The rescue mechanism operates on chunk *references* to article numbers, not on chunks that *define* those articles. In approximately 5 % of

rescue invocations, the surfaced chunk text mentions the queried article in passing (e.g., a cassation ruling cross-referencing the article) rather than providing the article's text. This is sufficient for validation but suboptimal for answer quality. A dedicated *article-definition* index, populated by paragraph-level segmentation of the Penal Code text, would address this and is planned for future work.

Index coverage and embedding model. The current index uses `paraphrase-multilingual-MiniLM-L12-v2` (384-dimensional). Aboasal et al. [5] report substantially higher MAP scores with BGE-M3 (1,024-dimensional) on a comparable Arabic legal task. An index rebuild with BGE-M3 plus article-aware chunking is the single largest remaining improvement available, but is gated on the 3–7 hour CPU rebuild time, which we defer to future work.

Evaluation scope. Our 28-question benchmark is comparatively small relative to the 13,000-case ALARB benchmark of [3] and the 2,500-pair dataset of [2]. While our benchmark was designed for fast, repeatable, manually-auditable evaluation during development, scaling to a comparably-sized benchmark for camera-ready evaluation is the natural next step.

---

8. Conclusion

We have presented **Conan**, a hallucination-resistant Arabic legal RAG system for Egyptian Criminal Law. By layering six grounding-defence mechanisms — input gating, citation-grounding prompts, evidence validation, corrective retry with a block-list, article-lookup rescue, iterative retrieval, and deterministic answer post-processing — on top of a standard hybrid-retrieval foundation, we drive the rate of hallucinated statutory citations from a baseline of 28.6 % down to **0.0 %** on a 28-question Egyptian-criminal-law benchmark. We additionally introduce a unified document-upload subsystem supporting three retention semantics across `.txt`, `.pdf`, and `.docx` formats. We release the system, the evaluation harness, and the five-version CSV evaluation trail to support reproducible auditing.

Our key methodological contribution is the demonstration that **no single mechanism is sufficient** for hallucination suppression in Arabic legal RAG; instead, a layered approach matching the heterogeneous failure modes of the underlying components (LLM memory citations, retrieval coverage gaps, fragmentary user inputs, wrong-law confusion, prompt-template leakage) is required. We hope this work informs and accelerates the deployment of Arabic legal-AI systems in production settings.

Acknowledgements

The authors thank the open-source community behind FAISS, BM25, BGE-Reranker-v2-M3, the Groq LLM platform, Google Gemini, and OpenRouter, without whose tools this work would not be possible.

References

[1] S. R. El-Beltagy and M. A. Abdallah, "Exploring Retrieval Augmented Generation in Arabic," *Procedia Computer Science*, vol. 244, pp. 296–307, 2024. *6th International Conference on AI in Computational Linguistics (ACLing 2024)*. DOI: 10.1016/j.procs.2024.10.203.

[2] J. Hrimech, M. Mghari, and Y. Zaz, "Retrieval-augmented generation for Arabic legal information: the family code case study," *TELKOMNIKA Telecommunication Computing Electronics and Control*, vol. 23, no. 6, pp. 1495–1505, Dec. 2025. DOI: 10.12928/TELKOMNIKA.v23i6.27400. ISSN: 1693-6930.

[3] H. Abu Shairah, S. AlHarbi, A. AlHussein, S. Alsabea, O. Shaqaqi, H. AlShamlan, O. Knio, and G. Turkiyyah, "ALARB: An Arabic Legal Argument Reasoning Benchmark," in *Proceedings of the Third Arabic Natural Language Processing Conference*, pp. 389–406, Association for Computational Linguistics, Nov. 8–9, 2025.

[4] M. Alghamdi, M. Abushawarib, M. Ellouh, M. Ghaleb, and M. Felemban, "Enhancing Arabic Information Retrieval for Question Answering," in *ICFNDS '23: Proceedings of the 7th International Conference on Future Networks and Distributed Systems*, Dubai, UAE, Dec. 21–22, 2023. DOI: 10.1145/3644713.3644763. ISBN: 9798400709036.

[5] R. Aboasal, S. Montasser, F. Hossam Eldin, A. Abdelwahab, and H. Abdelazim, "Arabic Legal Information Retrieval: The Impact of Morphological Segmentation and Semantic Embeddings," *Procedia Computer Science*, vol. 275, pp. 275–282, 2026. *7th International Conference on AI in Computational Linguistics (ACLing 2025)*.

[6] "Mini-RAG Project: Learnings & Technologies," internal RAG-implementation documentation, on file with the authors.

Manuscript prepared as a draft for a future conference submission. Source code, evaluation CSVs (versions v3–v8), and the evaluation harness are available alongside this document in the project repository.